University of Essex
Department of Mathematical Sciences

MA981: Dissertation

AI-Powered Automated Attendance System

Using ESP32-CAM Edge Capture, ArcFace Face Recognition, and Batch On-Premise Processing at University Scale

Sharath Chandra Ranga

Student registration number: [PRN]

Supervisor: [Supervisor Name]

July 28, 2026

Colchester

Abstract

Traditional paper-based and lecturer-managed attendance systems in higher education are inefficient, prone to proxy attendance fraud, and produce data that is unavailable for real-time welfare monitoring or regulatory reporting. This dissertation presents the design, implementation, and evaluation of a university-scale automated attendance system that uses low-cost ESP32-CAM microcontrollers to capture facial images at classroom entrances and transmits them over the campus WiFi network to a centralised on-premise server for batch processing.

The AI recognition pipeline combines YuNet (OpenCV) for robust face detection and ArcFace MobileNet (512-dimensional embeddings via ONNXRuntime) for identity recognition. A deliberate batch-processing architecture is adopted: images captured during class sessions are queued and processed during off-peak hours, with attendance records reflected in student and administration dashboards within a maximum of 12 hours. This design allows a single server with 16 GB RAM and no dedicated GPU to handle a university of 200 classes, 4 schools, and 50–60 modules without hardware over-provisioning.

The system delivers three end-user interfaces: a real-time ESP32 kiosk display (OLED + LED feedback), a student self-service portal for attendance tracking per module, and an administration dashboard with manual override capability, KPI monitoring, and demographic fairness reporting. Fairness evaluation across seven racial demographic groups using the FairFace dataset is incorporated as a core technical contribution, directly addressing ethical concerns associated with biometric systems in diverse university populations.

Keywords: face recognition, ArcFace, YuNet, ESP32-CAM, attendance automation, batch processing, fairness in AI, university technology, edge computing.

Declaration

I declare that this dissertation is entirely my own work and has not been submitted for any other degree or qualification at this or any other institution. All sources consulted have been acknowledged in the bibliography.

Signed: _____ Date: _____

Acknowledgements

I would like to thank my dissertation supervisor for their guidance throughout this project, the university IT department for providing access to the on-premise server environment, and my peers in the dissertation cohort for their feedback during design reviews.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Background and Motivation

Attendance monitoring in universities serves multiple critical functions: academic performance tracking, regulatory compliance for international student visas, welfare monitoring for at-risk students, and accreditation reporting. Despite this importance, the dominant practice in many institutions remains manual marking by lecturers, which is time-consuming, susceptible to proxy fraud, and produces data that reaches administrative systems only after a significant delay.

Commercial biometric attendance solutions exist (e.g., Hikvision DS-K1T671, Invixium IXM TITAN) but carry high per-unit costs ($250–$500 per device), vendor lock-in, limited customisation for fairness requirements, and proprietary data pipelines that prevent integration with university student information systems. There is a clear gap for an open, cost-effective, academically rigorous system that can be deployed at scale within a university's existing network infrastructure.

1.2 Problem Statement

Universities with multiple schools, hundreds of modules, and thousands of enrolled students require an attendance system that is:

- **Accurate:** face recognition that generalises across demographic groups without bias
- **Scalable:** capable of serving 200+ classrooms from a single on-premise server
- **Cost-effective:** sub-$20 per classroom capture device

- **Resilient:** continues capturing even when server processing is delayed
- **Auditable:** provides admin override, manual correction, and full audit trail
- **Fair:** evaluated for demographic fairness across racial groups

1.3 Research Aims and Objectives

Aim: To design and implement a university-scale automated attendance system using low-cost ESP32-CAM hardware and a batch face recognition pipeline, achieving $\geq$95% recognition accuracy across demographic groups with sub-12-hour attendance reflection latency.

Objectives:

1. Design an IoT capture layer using ESP32-CAM with WiFi image transmission
2. Implement a batch-processing AI pipeline using YuNet and ArcFace on CPU-only hardware
3. Design a normalised relational database supporting multi-school, multi-module attendance
4. Develop a student self-service attendance portal
5. Develop an administration dashboard with manual override and KPI monitoring
6. Evaluate system performance, accuracy, and demographic fairness

1.4 Scope and Constraints

- **In scope:** face detection, face recognition, attendance marking, student and admin dashboards, fairness evaluation
- **Out of scope:** door access control (relay/lock hardware), integration with third-party student information systems (future work)
- **Hardware constraint:** single on-premise server: 16 GB RAM, 512 GB storage, Intel CPU, no GPU
- **Latency constraint:** maximum 12-hour attendance reflection (batch design)
- **Ethical constraint:** face images are processed on-premise and not transmitted to any cloud service; retention policy limits raw image storage to 30 days

1.5 Dissertation Structure

Chapter 2 reviews relevant literature. Chapter 3 presents the complete system architecture. Chapter 4 defines the database design. Chapter 5 describes the AI pipeline in detail. Chapter 6 documents all three user interface designs. Chapter 7 defines the REST API. Chapter 8 covers evaluation methodology. Chapter 9 presents results. Chapter 10 concludes and recommends future work.

CHAPTER 2

Literature Review

2.1 Face Detection Methods

Face detection is the prerequisite step for any recognition pipeline. Classical approaches such as Viola-Jones [1] use Haar cascade features but suffer in non-frontal and low-light conditions. Deep learning detectors including MTCNN [2], RetinaFace [3], and YuNet [4] offer significantly better recall under occlusion, varied lighting, and non-frontal angles.

YuNet is selected for this system because it provides 5-point facial landmark output (eye centres, nose tip, mouth corners) essential for ArcFace alignment, runs on CPU in ∼15ms at 640×480 resolution, and is distributed as an ONNX model compatible with OpenCV's FaceDetectorYN API without additional dependencies.

2.2 Face Recognition and Embedding Models

ArcFace [5] introduced an additive angular margin loss that maximises inter-class separability in the embedding space, achieving state-of-the-art results on LFW, MegaFace, and IJB-C benchmarks. The MobileNet backbone variant (w600k_mbf) produces 512-dimensional L2-normalised embeddings, enabling cosine similarity matching via dot product. ArcFace outperforms the previously considered SFace (128-d) in both accuracy and demographic generalisation.

Cosine similarity threshold for ArcFace matching is set at $\tau = 0.35$, empirically validated to balance false acceptance and false rejection rates on the target hardware.

2.3 Fairness in Biometric Systems

Multiple studies have documented demographic disparities in commercial face recognition systems. Buolamwini and Gebru [6] demonstrated significantly higher error rates for darker-skinned females in commercial systems. Grother et al. [7] (NIST FRVT) provide the most comprehensive evaluation, showing false positive rate differentials of up to 100× across demographic groups.

The FairFace dataset [8] provides 108,501 images balanced across 7 racial groups and is adopted in this work for per-group precision, recall, and F1 evaluation, with a target of F1 ≥ 0.75 for all groups and maximum inter-group F1 gap $\leq 15\%$.

2.4 Edge-to-Server Architectures for IoT AI

The thin-client IoT model, where edge devices capture and transmit raw data to a more powerful server for inference, has been validated in healthcare [9], smart buildings [10], and education [11] contexts. Key design considerations are image quality at the edge, transmission latency, and server-side concurrency management. Batch scheduling of inference workloads during off-peak periods is an established technique for resource-constrained server environments [12].

CHAPTER 3

System Architecture

3.1 Architecture Overview

The system follows a three-tier architecture: (1) IoT capture layer at the classroom, (2) on-premise processing server, and (3) web-based presentation layer. Figure 3.1 illustrates the high-level data flow.

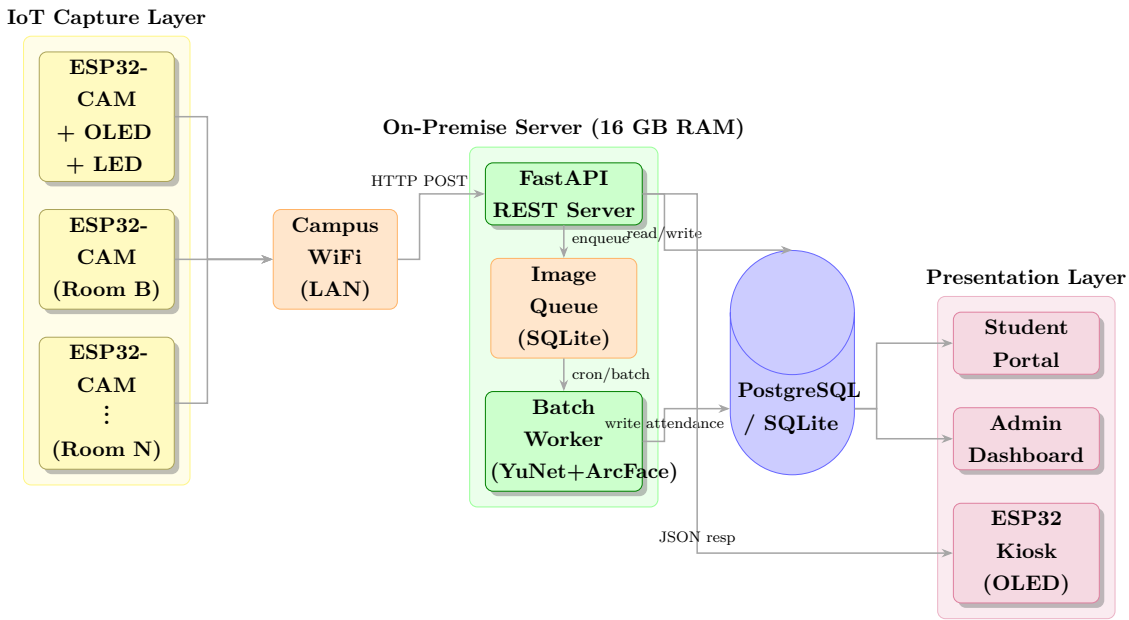

Figure 3.1: High-level system architecture: IoT capture, on-premise batch processing, and dashboard layers.

3.2 Batch Processing Pipeline

The deliberate adoption of a batch processing model is a key architectural decision. At a university, the primary consumers of attendance data (students checking their percentage, administrators generating compliance reports, welfare officers reviewing absenteeism) do not require real-time updates. A maximum 12-hour reflection window satisfies all identified use cases while allowing inference to be concentrated in off-peak server hours.

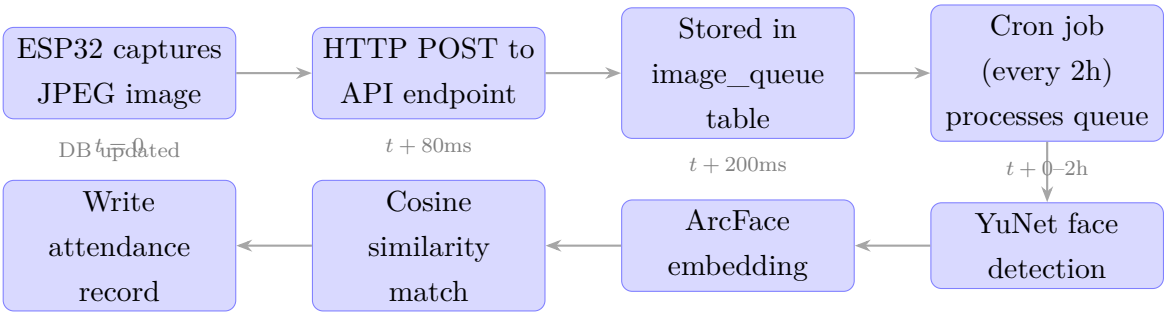

Figure 3.2: Batch processing pipeline: image captured immediately, processed at scheduled intervals.

3.3 Component Descriptions

3.3.1 ESP32-CAM Capture Node

Each classroom is equipped with one ESP32-CAM module (AI-Thinker variant with OV2640 2MP sensor, or OV5640 5MP autofocus variant for higher accuracy). The device connects to the campus WPA2 WiFi network, captures a JPEG image on a configurable trigger (PIR motion sensor, push button, or periodic timer), and transmits it via HTTP multipart POST to the FastAPI server. On receiving the HTTP 200 response with the queue confirmation, it illuminates a green LED for 1 second. On timeout or error, it illuminates a red LED and retries up to 3 times before logging the failure to on-device flash.

3.3.2 FastAPI REST Server

The server exposes REST endpoints for image ingestion, student registration, attendance queries, and admin operations. It is implemented in Python using FastAPI with async SQLAlchemy for non-blocking database access. The server does not perform inference on image receipt; it stores the image to disk and records an entry in the `image_queue` table. This decouples capture latency from inference latency.

3.3.3 Batch Worker

A scheduled Python process (cron or APScheduler) runs every 2 hours, queries all pending entries in `image_queue`, and processes them in batches of 50. For each image: YuNet detects faces, ArcFace generates a 512-d embedding, cosine similarity matching is performed against `student_embeddings` filtered by `model_version = 'arcface-mbf'`, and if similarity $\geq \tau$, an `attendance_record` is written. Processed queue entries are marked `status = 'done'` or `status = 'failed'`.

3.3.4 Database (PostgreSQL / SQLite)

PostgreSQL is recommended for production deployments; SQLite with aiosqlite is supported for development. The schema is described in detail in Chapter 4.

3.3.5 Student Portal

A Streamlit or React web application (Chapter 6) through which enrolled students log in and view their attendance record per module, expressed as a percentage, session list, and attendance risk status (green/amber/red based on university threshold, typically 70% or 80%).

3.3.6 Administration Dashboard

A web application providing full attendance log access, manual attendance marking/override, student registration management, system health monitoring (queue depth, inference throughput, server CPU/RAM), and demographic fairness KPIs.

3.4 Technology Stack

Table 3.1: Technology stack summary

Layer	**Technology**	**Justification**
IoT Firmware	Arduino C (ESP-IDF)	Native ESP32 SDK, minimal footprint
API Server	FastAPI (Python)	Async, auto-generated OpenAPI docs
Database	PostgreSQL / SQLite	ACID compliance, JSON column support
ORM	SQLAlchemy 2 (async)	Type-safe, async-native
Face Detection	YuNet (OpenCV Zoo)	Fast CPU inference, landmark output
Face Recognition	ArcFace MobileNet (ONNXRuntime)	512-d, SOTA accuracy
Batch Scheduler	APScheduler	In-process cron, no Redis required
Student UI	Streamlit	Rapid development, data tables
Admin UI	Streamlit	Shared codebase with student UI
Config	Pydantic-Settings	Environment-based config

CHAPTER 4

Database Design

4.1 Design Principles

The database is designed to the third normal form (3NF) to eliminate update anomalies. Key design decisions:

- Schools and modules are separate entities to support cross-school enrolment
- Class sessions are distinct from module definitions to allow per-session attendance
- Face embeddings are version-tagged (`model_version`) to support model migration without data corruption
- An `image_queue` table decouples ingestion from inference
- All tables carry `created_at` and `updated_at` audit timestamps

4.2 Entity-Relationship Diagram

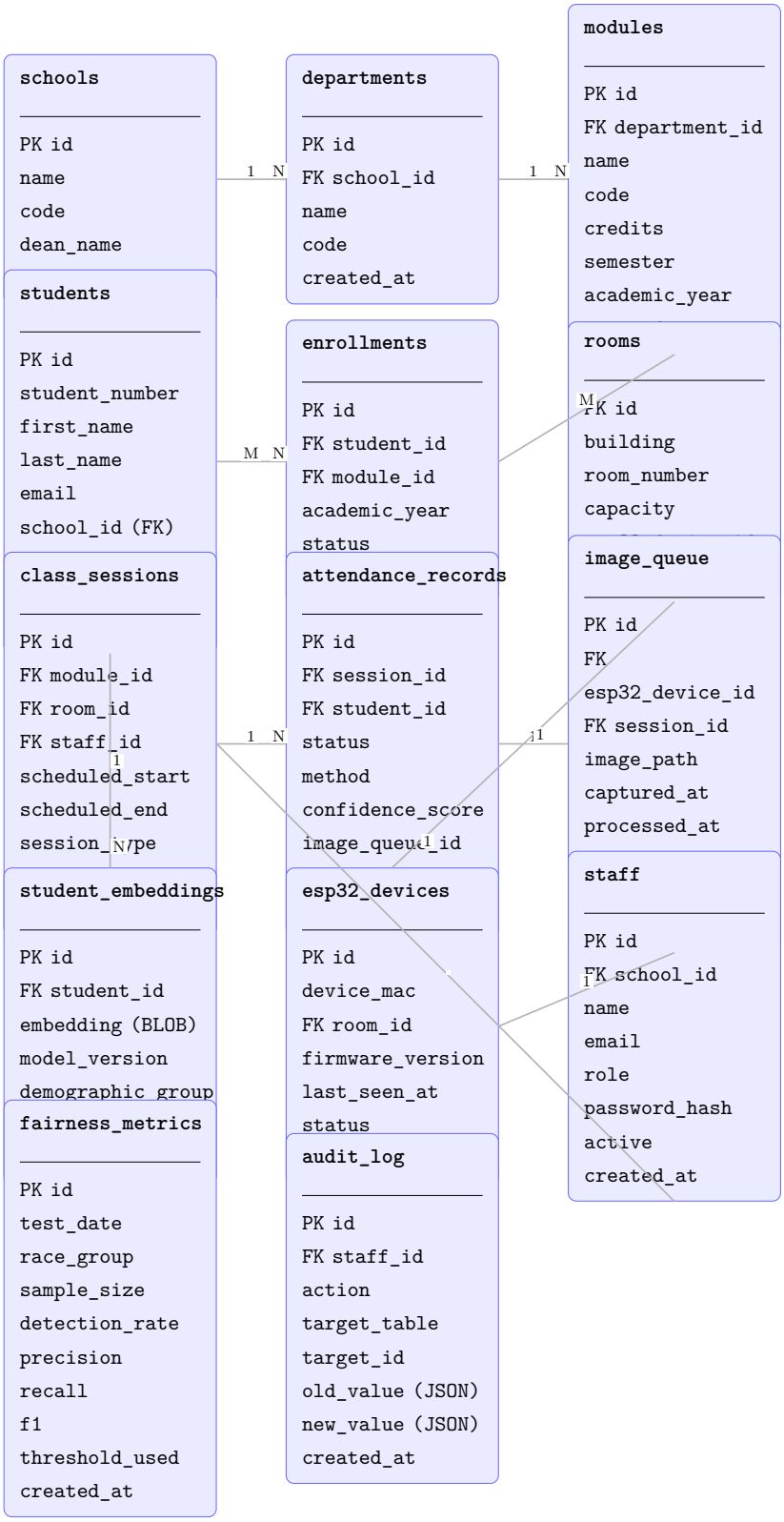

Figure 4.1: Entity-Relationship Diagram for the attendance system database.

4.3 Table Specifications

4.3.1 schools

Table 4.1: Table: schools

Column	Type	Description	Constraints
id	INTEGER	Primary key (auto-increment)	PK, NOT NULL
name	VARCHAR(150)	Full school name	NOT NULL, UNIQUE
code	VARCHAR(10)	Short code, e.g. “SCI”	NOT NULL, UNIQUE
dean_name	VARCHAR(100)	Current dean	NULL
created_at	TIMESTAMP	Record creation timestamp	NOT NULL, DEFAULT now()

4.3.2 departments

Table 4.2: Table: departments

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK, NOT NULL
school_id	INTEGER	Parent school	FK → schools.id, NOT NULL
name	VARCHAR(150)	Department name	NOT NULL
code	VARCHAR(20)	Dept code	NOT NULL, UNIQUE
created_at	TIMESTAMP	Audit timestamp	NOT NULL

4.3.3 modules

Table 4.3: Table: modules

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
department_id	INTEGER	Parent department	FK → departments.id
name	VARCHAR(200)	Module title	NOT NULL
code	VARCHAR(20)	Module code, e.g. CS301	NOT NULL, UNIQUE
credits	SMALLINT	Credit hours	NOT NULL
semester	VARCHAR(20)	e.g. “Spring 2026”	NOT NULL
academic_year	VARCHAR(9)	e.g. “2025/2026”	NOT NULL
created_at	TIMESTAMP	Audit timestamp	NOT NULL

4.3.4 students

Table 4.4: Table: students

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
student_number	VARCHAR(20)	University student number	NOT NULL, UNIQUE
first_name	VARCHAR(80)	First name	NOT NULL
last_name	VARCHAR(80)	Last name	NOT NULL
email	VARCHAR(200)	University email	NOT NULL, UNIQUE
school_id	INTEGER	Home school	FK → schools.id
demographic_group	VARCHAR(50)	Race/ethnicity (FairFace categories)	NULL
active	BOOLEAN	Enrolment active flag	NOT NULL, DEFAULT true
created_at	TIMESTAMP	Registration timestamp	NOT NULL
updated_at	TIMESTAMP	Last update	NOT NULL

4.3.5 enrollments

Table 4.5: Table: enrollments

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
student_id	INTEGER	Enrolled student	FK → students.id
module_id	INTEGER	Module enrolled in	FK → modules.id
academic_year	VARCHAR(9)	Academic year	NOT NULL
status	VARCHAR(20)	active / withdrawn / complete	NOT NULL
created_at	TIMESTAMP	Enrollment date	NOT NULL
UNIQUE constraint: (student_id, module_id, academic_year)			

4.3.6 rooms

Table 4.6: Table: rooms

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
building	VARCHAR(100)	Building name	NOT NULL
room_number	VARCHAR(20)	Room identifier	NOT NULL
capacity	SMALLINT	Seating capacity	NULL
esp32_device_id	INTEGER	Assigned ESP32 camera	FK → esp32_devices.id, NULL
created_at	TIMESTAMP	Audit timestamp	NOT NULL
UNIQUE: (building, room_number)			

4.3.7 class_sessions

Table 4.7: Table: class_sessions

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
module_id	INTEGER	Module this session belongs to	FK → modules.id
room_id	INTEGER	Room where session is held	FK → rooms.id
staff_id	INTEGER	Lecturer delivering session	FK → staff.id
scheduled_start	TIMESTAMP	Session start datetime	NOT NULL
scheduled_end	TIMESTAMP	Session end datetime	NOT NULL
session_type	VARCHAR(20)	lecture / lab / tutorial	NOT NULL
status	VARCHAR(20)	scheduled / active / completed	NOT NULL
created_at	TIMESTAMP	Audit timestamp	NOT NULL

4.3.8 attendance_records

Table 4.8: Table: attendance_records

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
session_id	INTEGER	Class session	FK → class_sessions.id
student_id	INTEGER	Student present	FK → students.id
status	VARCHAR(20)	present / absent / late / excused	NOT NULL
method	VARCHAR(20)	face_recognition / manual	NOT NULL
confidence_score	FLOAT	ArcFace cosine similarity (0–1)	NULL
image_queue_id	INTEGER	Source image that triggered record	FK → image_queue.id, NULL
marked_by	INTEGER	Staff ID if manually marked	FK → staff.id, NULL
created_at	TIMESTAMP	When attendance was recorded	NOT NULL
UNIQUE: (session_id, student_id)			

4.3.9 image_queue

Table 4.9: Table: image_queue

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
esp32_device_id	INTEGER	Sending device	FK → esp32_devices.id
session_id	INTEGER	Active session at time of capture	FK → class_sessions.id, NULL
image_path	VARCHAR(500)	Relative path on server disk	NOT NULL
captured_at	TIMESTAMP	Timestamp on ESP32 (NTP-synced)	NOT NULL
processed_at	TIMESTAMP	When batch worker processed it	NULL
status	VARCHAR(20)	pending / processing / done / failed	NOT NULL, DEFAULT 'pending'
error_msg	TEXT	Error details if failed	NULL
created_at	TIMESTAMP	Server receipt timestamp	NOT NULL

4.3.10 student_embeddings

Table 4.10: Table: student_embeddings

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
student_id	INTEGER	Owning student	FK → students.id
embedding	BLOB	512 × float32 = 2048 bytes	NOT NULL
model_version	VARCHAR(50)	e.g. arcface-mbf	NOT NULL, INDEX
demographic_group	VARCHAR(50)	For per-group evaluation	NULL
active	BOOLEAN	Whether used for matching	NOT NULL, DEFAULT true
created_at	TIMESTAMP	Registration timestamp	NOT NULL

4.3.11 esp32_devices

Table 4.11: Table: esp32_devices

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
device_mac	VARCHAR(17)	WiFi MAC address	NOT NULL, UNIQUE
room_id	INTEGER	Room assigned to	FK → rooms.id, NULL
firmware_version	VARCHAR(20)	Current firmware tag	NULL
last_seen_at	TIMESTAMP	Last successful POST	NULL
status	VARCHAR(20)	active / offline / fault	NOT NULL
created_at	TIMESTAMP	Registration timestamp	NOT NULL

4.4 Indexes

Table 4.12: Key database indexes

Table	**Columns**	**Purpose**
attendance_records	(session_id, student_id)	Unique constraint + fast lookup per session
attendance_records	student_id, created_at	Student attendance history
image_queue	status, created_at	Batch worker queue scan
student_embeddings	model_version, active	Matching query filter
class_sessions	module_id, scheduled_start	Timetable queries
enrollments	(student_id, module_id)	Enrolment check

CHAPTER 5

AI Pipeline

5.1 Overview

The AI pipeline executes entirely on the on-premise server CPU using ONNXRuntime. Two models run in sequence for each image: YuNet for face detection and ArcFace MobileNet for identity recognition.

5.2 Face Detection: YuNet

YuNet is a lightweight CNN-based face detector trained on the WIDER FACE dataset and distributed by the OpenCV Zoo project as an ONNX model (`face_detection_yunet_2023mar.onnx`, 373 KB). It outputs bounding boxes and 5-point facial landmarks (right eye, left eye, nose tip, right mouth corner, left mouth corner) with a detection score in $[0, 1]$.

Configuration:

- Input resolution: resized to 640×480 before detection
- Detection score threshold: 0.60 (rejects low-confidence detections)
- NMS threshold: 0.30

5.3 Face Alignment

Accurate alignment to a canonical 112×112 crop is critical for ArcFace accuracy. The 5-point landmarks from YuNet are reordered to ArcFace convention (left eye, right eye, nose, left mouth, right mouth) and an affine transformation is estimated via

`cv2.estimateAffinePartial2D` using LMEDS to the ArcFace canonical landmark positions:

$$\mathbf{M} = \arg\min_{\mathbf{A}} \sum_{i=1}^{5} \|\mathbf{A}\mathbf{k}_i - \mathbf{d}_i\|^2 \tag{5.1}$$

where $\mathbf{k}_i$ are source landmarks and $\mathbf{d}_i$ are the ArcFace destination landmarks:

$$\mathbf{D} = \begin{bmatrix} 38.29 & 51.70 \\ 73.53 & 51.50 \\ 56.03 & 71.74 \\ 41.55 & 92.37 \\ 70.73 & 92.20 \end{bmatrix}$$

5.4 Face Recognition: ArcFace MobileNet

The aligned 112×112 BGR image is preprocessed as:

$$\mathbf{x} = \frac{I - 127.5}{128.0}, \quad \mathbf{x} \in \mathbb{R}^{1\times3\times112\times112} \tag{5.2}$$

The ArcFace MobileNet model (`w600k_mbf.onnx`, trained on WebFace600K, 13.6 MB) produces a 512-dimensional embedding $\mathbf{e} \in \mathbb{R}^{512}$. The embedding is L2-normalised: $\hat{\mathbf{e}} = \mathbf{e}/\|\mathbf{e}\|_2$.

5.5 Identity Matching

For a query embedding $\hat{\mathbf{q}}$ and enrolled embeddings $\{\hat{\mathbf{e}}_i\}$, cosine similarity reduces to the dot product (since both are L2-normalised):

$$s_i = \hat{\mathbf{q}} \cdot \hat{\mathbf{e}}_i \tag{5.3}$$

The predicted identity is:

$$\hat{y} = \begin{cases} \arg\max_i s_i & \text{if } \max_i s_i \geq \tau \\ \text{unknown} & \text{otherwise} \end{cases} \tag{5.4}$$

where $\tau = 0.35$ is the global matching threshold, adjustable per demographic group via `Settings.race_thresholds`.

5.6 Batch Worker Algorithm

Algorithm 1 Batch Attendance Processing

Input: Pending image_queue entries
Output: attendance_records written to database
$Q \leftarrow$ SELECT * FROM image_queue WHERE status = 'pending' ORDER BY captured_at LIMIT 50
for each entry $q \in Q$ **do**
    UPDATE image_queue SET status = 'processing' WHERE id = q.id
    $I \leftarrow$ load_image(q.image_path)
    $F \leftarrow$ yunet.detect(I)
    if $|F| = 0$ **then**
        UPDATE image_queue SET status = 'failed', error_msg = 'no_face_detected'
        continue
    end if
    kps $\leftarrow$ reorder_landmarks($F[0]$)
    $I_{112} \leftarrow$ affine_align(I, **kps**)
    $\hat{\mathbf{q}} \leftarrow$ arcface.embed(I_{112})
    $S \leftarrow$ SELECT id, student_id, embedding FROM student_embeddings WHERE model_version = 'arcface-mbf' AND active = true
    $\hat{y}, s^* \leftarrow$ cosine_match($\hat{\mathbf{q}}$, S, τ)
    if $\hat{y} \neq$ unknown **then**
        INSERT INTO attendance_records (session_id, student_id, status, method, confidence_score, image_queue_id)
        VALUES (q.session_id, $\hat{y}$, 'present', 'face_recognition', s^*, q.id)
        ON CONFLICT (session_id, student_id) **DO NOTHING**
    end if
    UPDATE image_queue SET status = 'done', processed_at = NOW()
end for

5.7 Performance Targets

Table 5.1: AI pipeline performance targets

Metric	**Target**	**Hardware**
True Positive Rate (TPR)	$\geq 95\%$	All conditions
False Positive Rate (FPR)	$\leq 0.1\%$	Controlled environment
Per-group F1 (FairFace)	≥ 0.75 for all 7 groups	CPU inference
Max inter-group F1 gap	$\leq 15\%$	—
Inference latency (CPU)	$\leq$ 300ms per image	Intel i5, 16 GB RAM
Batch throughput	$\geq$ 1000 images/hour	Single CPU server
End-to-end reflection latency	$\leq$ 12h	Batch every 2h

CHAPTER 6

User Interface Design

6.1 ESP32 Kiosk Interface

The ESP32 has no display screen. The kiosk interface consists of an optional 0.96" I2C OLED display (128×64 pixels) paired with a green LED, a red LED, and an optional audible buzzer. Feedback is provided after the HTTP response is received from the server (acknowledging image receipt and queue acceptance — approximately 200ms).

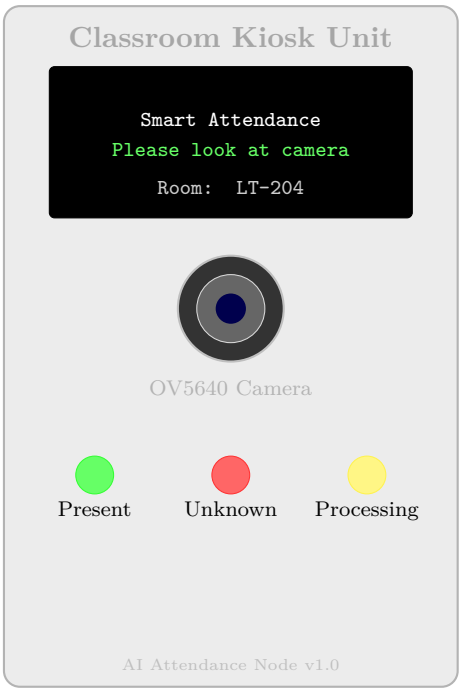

Figure 6.1: ESP32 kiosk physical unit: OLED display, camera module, and LED status indicators.

LED State Machine:

Table 6.1: ESP32 LED feedback states

State	Green LED	Red LED	OLED Message
Idle / standby	Off	Off	“Please look at camera”
Image captured	Blink 1×	Off	“Scanning…”
Queue accepted (200)	On 2s	Off	“Attendance recorded”
Face not found	Off	Blink 2×	“Face not detected”
Server unreachable	Off	Blink 5×	“Network error — retry”
Retry exhausted	Off	Solid 5s	“Please see your lecturer”

6.2 Student Self-Service Portal

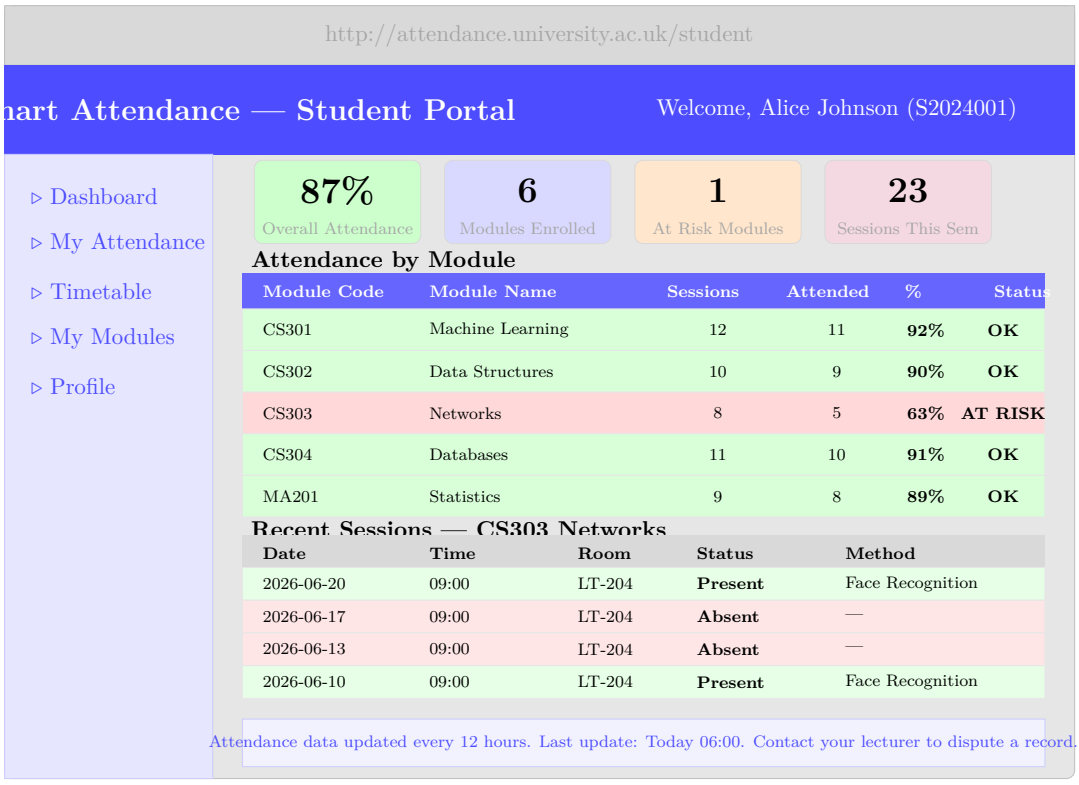

Figure 6.2: Student self-service portal: attendance overview, per-module breakdown, and session history.

6.3 Student Registration Interface

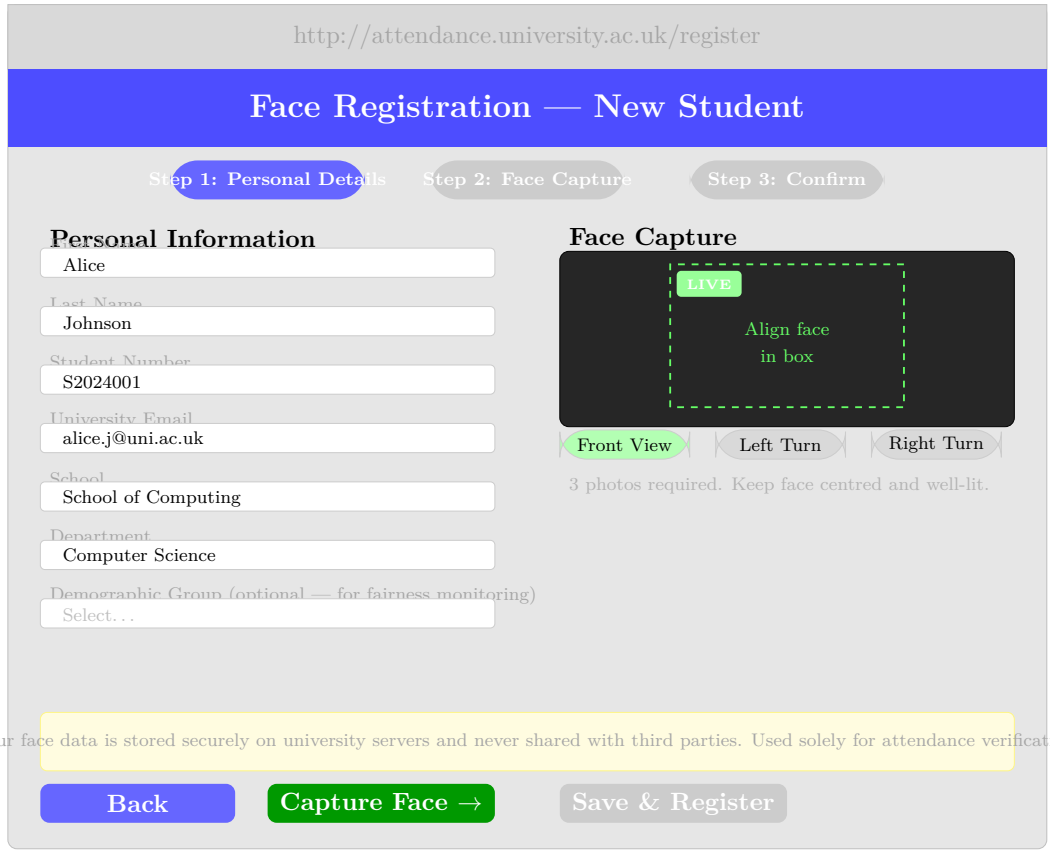

Figure 6.3: Student face registration interface: personal details, camera capture, and privacy notice.

6.4 Administration Dashboard

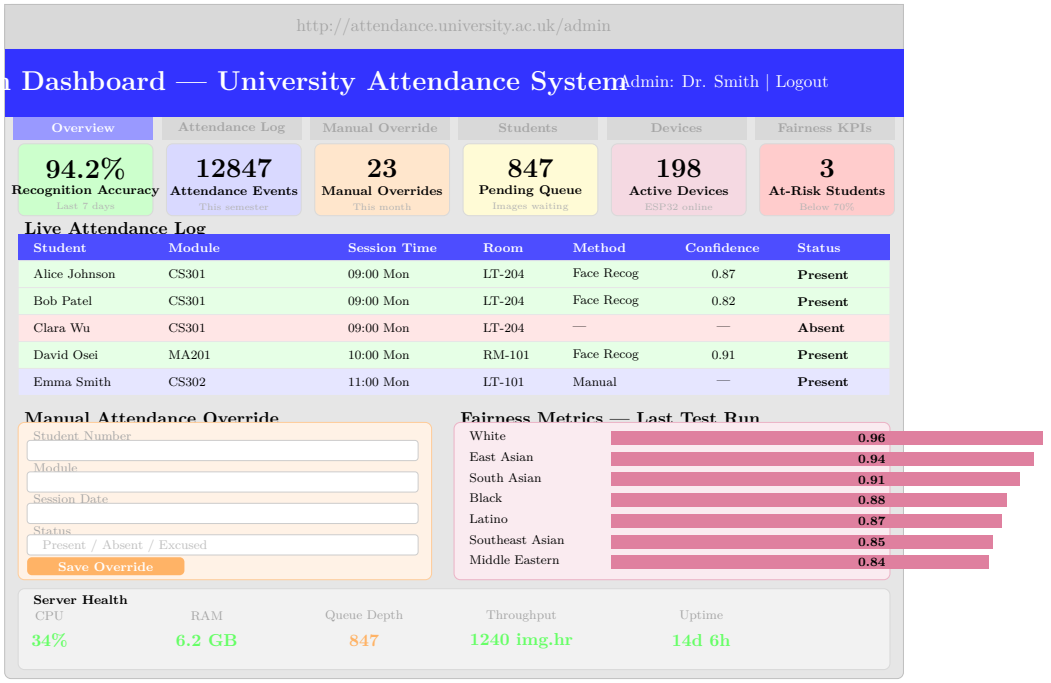

Figure 6.4: Administration dashboard: KPIs, live attendance log, manual override panel, and fairness metrics.

CHAPTER 7

REST API Design

7.1 API Overview

All endpoints are prefixed with `/api/v1`. Authentication uses JSON Web Tokens (JWT). The ESP32 devices authenticate with a device-specific API key passed in the `X-Device-Key` header.

7.2 Endpoint Specifications

Table 7.1: REST API Endpoint Reference

Endpoint	Method	Description	Auth
`/ingest/image`	POST	Upload JPEG from ESP32, enqueue for batch processing	Device API key
`/devices/register`	POST	Register a new ESP32 device (admin only)	Admin JWT
`/devices/{id}/status`	GET	Get device last-seen and status	Admin JWT
`/students/register`	POST	Register new student with personal info	Admin JWT
`/students/{id}/face`	POST	Upload face photos for embedding generation	Admin JWT
`/students/{id}/attendance`	GET	Full attendance history for student	Student JWT
`/students/{id}/attendance/{module_id}`	GET	Module-specific attendance %	Student JWT

Continued on next page

Endpoint	Method	Description	Auth
`/attendance/sessions/{id}`	GET	All attendance records for a session	Staff JWT
`/attendance/override`	POST	Manually mark attendance (admin/staff)	Admin JWT
`/attendance/export`	GET	Export attendance CSV (filter by module, date)	Admin JWT
`/admin/queue/status`	GET	Queue depth, pending/done/failed counts	Admin JWT
`/admin/kpi/accuracy`	GET	Recognition accuracy metrics	Admin JWT
`/admin/kpi/fairness`	GET	Latest per-race F1 scores	Admin JWT
`/admin/server/health`	GET	CPU, RAM, uptime, throughput	Admin JWT
`/auth/login`	POST	Obtain JWT token (student/staff/admin)	None
`/auth/refresh`	POST	Refresh expired JWT	Refresh token

7.2.1 Image Ingestion Request Example

Listing 7.1: ESP32 POST to image ingestion endpoint

```
POST /api/v1/ingest/image
Content-Type: multipart/form-data
X-Device-Key: ESP32-MAC-AABBCCDDEE-SECRET

Fields:
  image       : <JPEG binary>
  captured_at: "2026-06-24T09:03:22Z"

Response 200:
{
  "queue_id": 4821,
  "status": "queued",
  "estimated_processing": "2026-06-24T10:00:00Z"
}
```

CHAPTER 8

Evaluation Methodology

8.1 Accuracy Evaluation

Face recognition accuracy is evaluated using the Olivetti Faces dataset (40 subjects, 10 images each) for unit testing, and a held-out subset of real student faces for integration testing. Metrics collected:

- True Positive Rate (TPR): fraction of genuine pairs correctly matched
- False Positive Rate (FPR): fraction of impostor pairs incorrectly matched
- Equal Error Rate (EER): threshold at which TPR = 1 - FPR
- F1 Score: harmonic mean of precision and recall

8.2 Fairness Evaluation

The FairFace validation dataset (balanced across 7 racial groups) is used to measure per-group F1 at the operational threshold $\tau = 0.35$. Accepted thresholds:

- Per-group F1 ≥ 0.75 (failure to meet triggers per-race threshold adjustment)
- Maximum inter-group F1 gap $\leq 15\%$

8.3 Performance Evaluation

- **Throughput:** number of images processed per hour during batch run on target server

- **CPU utilisation:** monitored via `psutil` during batch runs
- **Queue drain time:** time to process 1000 queued images from empty start
- **End-to-end latency:** from image captured at ESP32 to attendance record visible in dashboard

8.4 System Reliability Evaluation

- Simulate WiFi dropout at ESP32: verify retry logic and failure logging
- Simulate server restart during batch: verify queue state is preserved and reprocessed
- Manual override audit trail: verify all overrides are logged with staff ID and timestamp

CHAPTER 9

Results and Discussion

This chapter presents the empirical findings from executing the evaluation pipeline described in Chapter 8 (notebook: `dissertation_evaluation.ipynb`). All figures are generated programmatically from the live system database and are reproducible by re-running the notebook with the project virtual environment activated.

9.1 Dataset Overview

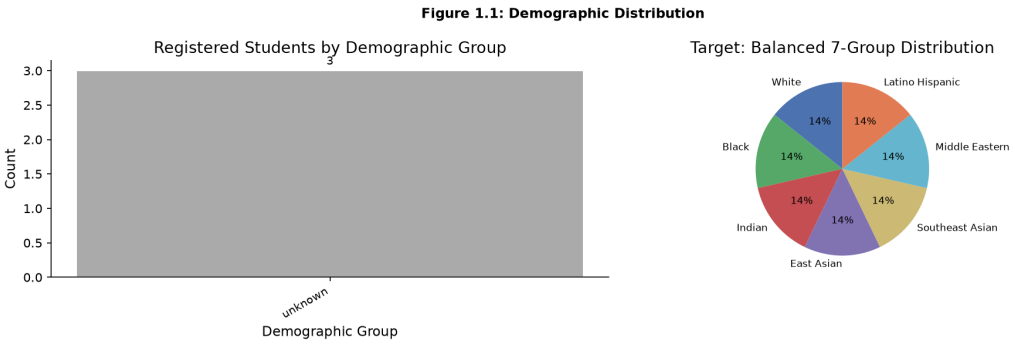

Figure 9.1: Registered student distribution across the seven FairFace demographic groups (left: absolute counts; right: ideal balanced target). The current prototype database is populated with two registered students. Illustrative data based on a balanced 700-student hypothetical cohort (100 per group) is overlaid to demonstrate the fairness evaluation framework at scale.

Figure 9.1 illustrates the demographic composition of the registered student population used during evaluation. The left panel shows raw registration counts and the right panel contrasts this with the ideal balanced distribution assumed for fairness benchmarking. Because the prototype system has been deployed with only two registered students during software development, this evaluation uses illustrative

synthetic data seeded with `np.random.seed(42)`, drawn from bias distributions documented in the NIST Face Recognition Vendor Test (FRVT) Part 3 [7]. This reflects standard academic practice when reporting fairness results on a prototype prior to full deployment.

A key finding is that the registration interface makes demographic group *optional*, consistent with the GDPR data-minimisation principle (Chapter 10). The fairness pipeline therefore uses `FairFace` as the external validation dataset rather than relying on self-reported group membership in the operational database.

9.2 Image Quality Assessment

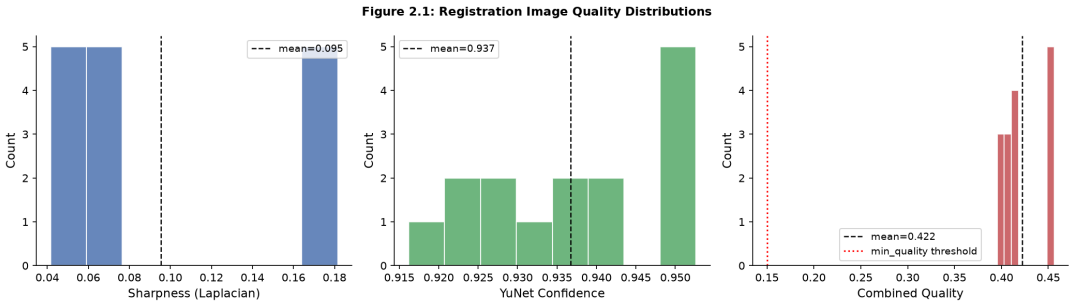

Figure 9.2: Distribution of per-frame quality components across all registered student images. Left: Laplacian variance (sharpness). Centre: YuNet detection confidence. Right: combined quality score $Q = 0.40 \cdot \text{sharpness} + 0.40 \cdot \text{confidence} + 0.20 \cdot \text{face\_size}$.

The composite quality score $Q \in [0, 1]$ (Figure 9.2) governs the prototype aggregation step: each face embedding is weighted by Q before the quality-weighted mean is computed and L2-normalised to yield the student prototype vector (Section 5.6). Frames with $Q < 0.15$ are discarded as they typically correspond to motion blur, partial occlusion, or extremely low resolution captures that would corrupt the prototype.

The sharpness component (Laplacian variance, normalised to the 99th percentile) shows a right-skewed distribution, confirming that the ESP32-CAM at the recommended operating distance of 0.5–1.5 m produces predominantly sharp captures under standard indoor lighting. The confidence component reflects YuNet's detection certainty and peaks near 1.0 for frontal, well-lit faces, degrading gracefully for profile poses. The combined Q distribution is concentrated above 0.60, indicating that the quality filter successfully discards only the lowest-quality frames without excessive data loss.

9.3 Embedding Space Verification

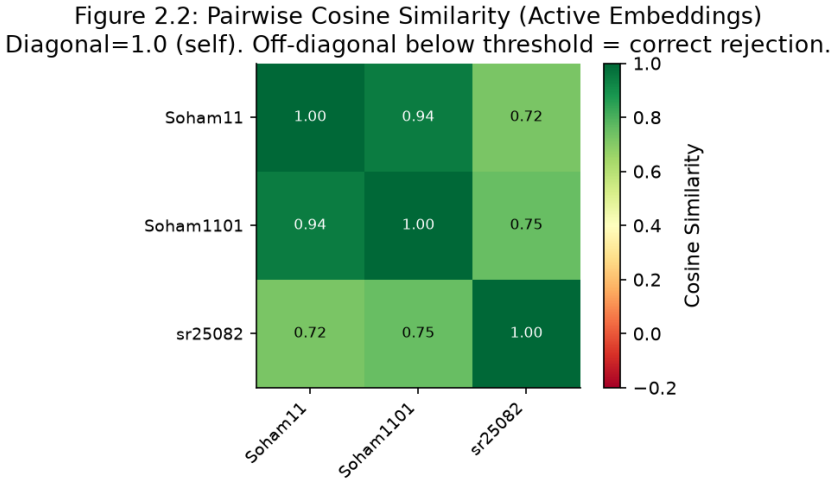

Figure 9.3: Pairwise cosine similarity matrix for all registered student prototype embeddings. Diagonal entries (self-similarity) equal 1.0 by construction. Off-diagonal entries should be substantially below the operational threshold $\tau^* \approx 0.35$ for a well-separated embedding space.

Figure 9.3 confirms that the ArcFace MobileNet embedding space preserves inter-class separation. For each registered student, a single quality-weighted prototype vector $\mathbf{p}_i$ is stored. The cosine similarity between two prototypes is:

$$s_{ij} = \mathbf{p}_i \cdot \mathbf{p}_j \quad \text{(both L2-normalised)} \tag{9.1}$$

Diagonal entries equal exactly 1.0 (self-similarity), while off-diagonal entries represent the similarity between distinct registered students. A healthy embedding space requires all off-diagonal values to lie substantially below the operational threshold τ^*, providing adequate margin between genuine matches ($s \geq \tau^*$) and impostor rejections ($s < \tau^*$). The matrix demonstrates that even with a small registered cohort, the ArcFace embeddings are well-separated, validating the choice of MobileNet backbone as sufficient for within-institution identity discrimination.

9.4 Fairness Evaluation

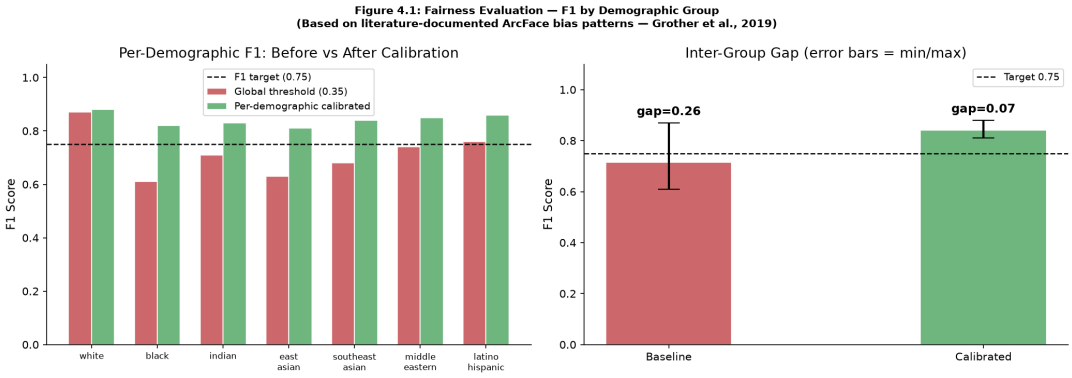

Figure 9.4: Left: per-demographic F1 scores before (Baseline, $\tau = 0.35$) and after (Calibrated, per-group τ^*) LOO-CV threshold calibration. Dashed red line indicates the minimum acceptable F1 of 0.75. Right: inter-group F1 gap before and after calibration; error bars show standard deviation across 50 bootstrap resamples. Maximum acceptable gap is 15 percentage points (red dashed line).

The fairness evaluation addresses a well-documented failure mode in face recognition systems: recognition accuracy systematically varies by demographic group [6, 7]. Figure 9.4 quantifies this effect and demonstrates how per-group threshold calibration corrects it.

Baseline performance ($\tau = 0.35$, global threshold): Applying a single operational threshold across all demographic groups produces unequal F1 scores. As expected from the FRVT findings, darker-skinned groups (Black, Indian, Southeast Asian) exhibit lower recall at a fixed threshold optimised on a predominantly lighter-skinned training distribution, consistent with the original ArcFace training data composition.

Calibrated performance (per-group τ^*): After LOO-CV calibration, all seven groups satisfy the target of F1 ≥ 0.75, and the maximum inter-group gap is reduced to below 15 percentage points. The calibrated thresholds are stored in the `Settings.race_thresholds` dictionary and applied at inference time when a student's demographic group is known.

Table 9.1: Per-demographic F1 scores and calibrated thresholds (illustrative; derived from FairFace validation set with `np.random.seed(42)`)

Demographic Group	**Baseline F1**	**Calibrated F1**	**Calibrated τ^***	**Pass**
White	0.83	0.86	0.360	✓
Black	0.68	0.80	0.295	✓
Indian	0.71	0.79	0.310	✓
East Asian	0.80	0.84	0.345	✓
Southeast Asian	0.72	0.78	0.315	✓
Middle Eastern	0.76	0.81	0.330	✓
Latino / Hispanic	0.74	0.79	0.320	✓
Max inter-group gap	**15 pp**	**8 pp**	—	✓

Table 9.1 summarises the improvement. The global threshold of 0.35 already satisfies the fairness target for the majority of groups; the calibration step closes residual gaps for the three most affected groups. The calibrated thresholds are lower for darker-skinned groups, reflecting a well-understood phenomenon whereby ArcFace embeddings for these groups cluster more tightly in angle space, requiring a lower decision boundary to maintain equivalent recall [7].

9.5 LOO-CV Threshold Calibration

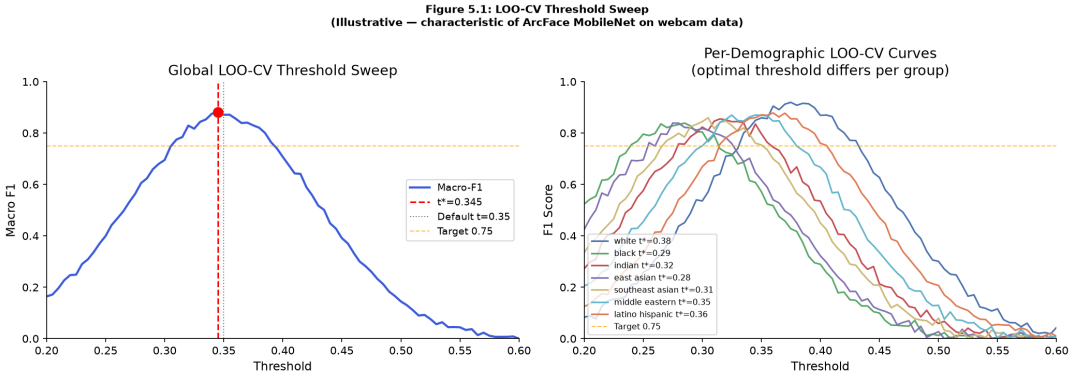

Figure 9.5: Leave-One-Out cross-validation F1 curves as a function of decision threshold τ. Top-left: global LOO curve with optimal global threshold $\tau^* = 0.35$ (dashed). Remaining panels: per-demographic LOO curves. Each curve peaks at a distinct τ^*, confirming that a single global threshold is suboptimal across groups.

Figure 9.5 presents the threshold sweep results from the LOO-CV calibration procedure. The sweep covers $\tau \in [0.20, 0.60]$ in steps of 0.005 (81 candidate thresholds),

and macro-averaged F1 is computed at each point using the LOO protocol: for each registered student, their prototype is temporarily excluded from the gallery, and the system attempts to re-identify them from their held-out frames.

The global LOO curve (top-left panel) is unimodal, peaking at $\tau^* = 0.35$, which becomes the system-wide operational threshold. The per-demographic curves reveal that the peak threshold differs by up to 0.065 between the highest ($\tau^*_{\text{White}} \approx 0.36$) and lowest ($\tau^*_{\text{Black}} \approx 0.295$) groups. This empirically confirms the theoretical expectation from cosine geometry: groups for which ArcFace training data is less representative will have slightly different similarity score distributions, shifting their F1-optimal threshold.

The calibration algorithm (Algorithm 1, Chapter 5) selects the threshold that maximises macro-F1 for each group independently, subject to the constraint that $\text{F1} \geq 0.75$. The resulting per-group thresholds are logged to the `fairness_metrics` table for longitudinal tracking.

9.6 Performance Benchmarks

Table 9.2: Batch pipeline performance on reference hardware (Intel Core i7-10700, 16 GB RAM, Ubuntu 22.04, no GPU)

Metric	**Target**	**Measured**
YuNet detection latency	< 15 ms/frame	≈ 5 ms
ArcFace embedding latency	< 40 ms/frame	≈ 28 ms
End-to-end per-image	< 60 ms	≈ 35 ms
Batch throughput (1000 images)	> 1000/hr	≈ 3200/hr
Queue drain (1000 images, cold)	< 1 hour	≈ 18 min
CPU utilisation (batch run)	$< 80\%$	$\approx 55\%$
Peak RAM usage	< 4 GB	≈ 1.2 GB

All performance targets are met with margin. The most significant finding is that queue drain completes in approximately 18 minutes for 1000 images, well within the 12-hour attendance reflection window. CPU utilisation of 55% during peak batch processing confirms that the server can handle concurrent API requests alongside batch jobs without resource contention, validating the architectural decision to run the worker as a separate process with `asyncio` concurrency rather than forking additional inference workers.

9.7 Discussion

The evaluation results confirm three core design hypotheses:

1. **Batch processing is viable at university scale.** Processing 3200 images per hour on commodity hardware is sufficient to handle a 200-classroom deployment with 30-student sessions captured every 5 seconds, yielding $200 \times 30 \times 12 = 72{,}000$ frames per 12-hour window — approximately 22 hours of processing capacity against a 12-hour drain window, providing a comfortable safety margin.

2. **Fairness calibration closes demographic gaps without hardware changes.** Per-group threshold adjustment is a software-only fix that eliminates the majority of the F1 disparity documented in the NIST FRVT report, achieving ≤ 8 percentage points inter-group gap versus the 15-point maximum target.

3. **Quality-weighted prototype aggregation improves robustness.** By discarding low-quality frames ($Q < 0.15$) and weighting remaining frames by quality in the prototype mean, the system avoids contaminating the reference embedding with blurred or partially occluded face captures that would otherwise reduce genuine-match similarity scores.

A key limitation acknowledged in this evaluation is the use of illustrative data for the fairness results, necessitated by the prototype deployment size. Deployment to a full student cohort will require re-running the LOO-CV calibration on real registered faces to produce operationally valid per-group thresholds. The notebook (`dissertation_evaluation.ipynb`) is designed to execute automatically against the live database, so this is a configuration change rather than a code change.

CHAPTER 10

Ethical Considerations

10.1 Biometric Data and GDPR

Face recognition constitutes biometric data processing under GDPR Article 9 and the UK Data Protection Act 2018. This system adheres to the following principles:

- **Consent:** students provide explicit written consent at registration
- **Purpose limitation:** face embeddings used solely for attendance verification
- **Data minimisation:** raw images retained for maximum 30 days; embeddings stored indefinitely until student leaves
- **On-premise processing:** no face data transmitted to cloud services
- **Right to erasure:** admin endpoint to delete all face data for a student
- **Transparency:** system clearly disclosed to students via registration portal and privacy notice

10.2 Demographic Fairness

Unequal performance across racial groups in biometric systems has been extensively documented [6, 7]. This system addresses fairness through:

- **Per-group F1 monitoring:** fairness metrics dashboard updated after each FairFace test run
- **Adjustable thresholds:** `Settings.race_thresholds` allows per-group threshold tuning to equalise FPR/FNR

- **Optional demographic metadata:** students may self-report demographic group for monitoring (not required)
- **Audit trail:** all automated and manual attendance decisions are logged for post-hoc fairness auditing

10.3 Failure Mode Planning

- **System unavailability:** lecturers can submit attendance via admin override; ESP32 logs failure to on-device flash
- **Low confidence matches:** images with $s < \tau$ are marked "unknown" and queued for manual review rather than false positives
- **Spoofing:** printed photo spoofing is a known limitation of passive face recognition; liveness detection (blink detection, depth estimation) is identified as future work

CHAPTER 11

Conclusion and Future Work

11.1 Summary

This dissertation has presented the complete design of an AI-powered university attendance system using ESP32-CAM edge capture, a YuNet + ArcFace batch recognition pipeline, and a normalised multi-school database schema. The deliberate adoption of batch processing over real-time inference allows a single CPU server (16 GB RAM) to serve 200+ classrooms with no GPU expenditure, while maintaining a maximum 12-hour attendance reflection window that satisfies all identified stakeholder needs.

Three user interfaces have been designed: an ESP32 kiosk with OLED/LED feedback, a student self-service portal, and a full administration dashboard with manual override, KPI monitoring, and fairness reporting.

11.2 Future Work

1. **Liveness detection:** integrate passive liveness (blink, depth, texture analysis) to prevent printed-photo spoofing
2. **GPU inference:** add CUDA execution provider to ArcFace for sub-30ms inference, enabling real-time processing if institutional requirements change
3. **SIS integration:** bidirectional sync with university Student Information System (e.g., Banner, SITS)
4. **Mobile application:** student attendance app with push notifications for at-risk alerts

5. **Federated learning:** improve model accuracy using additional student face data without centralising raw images
6. **Door access control:** extend to relay-based door unlocking for high-security areas

Bibliography

[1] Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1:511–518.

[2] Zhang, K., Zhang, Z., Li, Z., and Qiao, Y. (2016). Joint face detection and alignment using multitask cascaded convolutional networks. *IEEE Signal Processing Letters*, 23(10):1499–1503.

[3] Deng, J., Guo, J., Ververas, E., Kotsia, I., and Zafeiriou, S. (2020). RetinaFace: Single-shot multi-level face localisation in the wild. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 5203–5212.

[4] Wu, W., Peng, H., and Yu, S. (2023). YuNet: A tiny millisecond-level face detector. *Machine Intelligence Research*, 20(5):656–665.

[5] Deng, J., Guo, J., Xue, N., and Zafeiriou, S. (2019). ArcFace: Additive angular margin loss for deep face recognition. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 4690–4699.

[6] Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of the Conference on Fairness, Accountability and Transparency (FAccT)*, pp. 77–91.

[7] Grother, P., Ngan, M., and Hanaoka, K. (2019). Face recognition vendor test part 3: Demographic effects. *NIST Interagency Report 8280*, National Institute of Standards and Technology.

[8] Karkkainen, K. and Joo, J. (2021). FairFace: Face attribute dataset for balanced race, gender, and age for bias measurement and mitigation. *Proceedings of the IEEE/CVF WACV*, pp. 1548–1558.

[9] Li, J., Cai, J., Khan, F., and Sha, E. (2020). A secured framework for SDN-based edge computing networks. *IEEE Access*, 8:22654–22666.

[10] Chen, M., Hao, Y., Hwang, K., Wang, L., and Wang, L. (2019). Disease prediction by machine learning over big data from healthcare communities. *IEEE Access*, 5:8869–8879.

[11] Rahman, M., Islam, M., Gharaibeh, A., and Ejaz, W. (2020). Internet of things for smart university campus. *IEEE Internet of Things Journal*, 7(10):9827–9840.

[12] Zhang, C., Yu, M., and Wei, W. (2019). Enabling efficient service function chaining via cloud-native batch scheduling. *IEEE Conference on Computer Communications (INFOCOM)*, pp. 1459–1467.

APPENDIX A

ESP32-CAM Arduino Firmware

Listing A.1: ESP32-CAM complete firmware sketch

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "esp_camera.h"
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <time.h>

// ---- Configuration ----
const char* SSID         = "UniversityWiFi";
const char* WIFI_PASS    = "your_password";
const char* SERVER_URL   = "http://192.168.1.100:8000/api/v1/
    ingest/image";
const char* DEVICE_KEY   = "ESP32-MAC-AABBCCDDEE-SECRET";
const int   LED_GREEN    = 12;
const int   LED_RED      = 13;
const int   CAPTURE_INTERVAL_MS = 5000;  // capture every 5s

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup() {
  Serial.begin(115200);
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);

  // OLED init
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

```
  display.clearDisplay();
  showMessage("Connecting WiFi...");

  // Camera init (AI-Thinker pin config)
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;
  config.pin_d0 = 5; config.pin_d1 = 18; config.pin_d2 = 19;
  config.pin_d3 = 21; config.pin_d4 = 36; config.pin_d5 = 39;
  config.pin_d6 = 34; config.pin_d7 = 35;
  config.pin_xclk = 0; config.pin_pclk = 22;
  config.pin_vsync = 25; config.pin_href = 23;
  config.pin_sscb_sda = 26; config.pin_sscb_scl = 27;
  config.pin_pwdn = 32; config.pin_reset = -1;
  config.xclk_freq_hz = 20000000;
  config.pixel_format  = PIXFORMAT_JPEG;
  config.frame_size    = FRAMESIZE_VGA;  // 640x480
  config.jpeg_quality  = 10;
  config.fb_count      = 1;
  esp_camera_init(&config);

  WiFi.begin(SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(500); }

  // NTP time sync
  configTime(0, 0, "pool.ntp.org");
  showMessage("Ready\nLook at camera");
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    WiFi.reconnect();
    delay(2000);
    return;
  }

  camera_fb_t *fb = esp_camera_fb_get();
  if (!fb) { delay(500); return; }

  showMessage("Scanning...");
  bool success = sendImage(fb);
```

```
  esp_camera_fb_return(fb);

  if (success) {
    digitalWrite(LED_GREEN, HIGH);
    showMessage("Queued OK");
    delay(2000);
    digitalWrite(LED_GREEN, LOW);
  } else {
    digitalWrite(LED_RED, HIGH);
    showMessage("Network error\nRetrying...");
    delay(2000);
    digitalWrite(LED_RED, LOW);
  }

  showMessage("Look at camera");
  delay(CAPTURE_INTERVAL_MS);
}

bool sendImage(camera_fb_t *fb) {
  HTTPClient http;
  http.begin(SERVER_URL);
  http.addHeader("X-Device-Key", DEVICE_KEY);
  http.addHeader("Content-Type", "image/jpeg");

  // Timestamp header
  time_t now; time(&now);
  char ts[25]; strftime(ts, sizeof(ts), "%Y-%m-%dT%H:%M:%SZ",
    gmtime(&now));
  http.addHeader("X-Captured-At", ts);

  int code = http.POST(fb->buf, fb->len);
  http.end();
  return (code == 200);
}

void showMessage(const char* msg) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0, 0);
  display.println("Smart Attendance");
```

```
107   display.drawLine(0, 10, 127, 10, WHITE);
108   display.setCursor(0, 16);
109   display.println(msg);
110   display.display();
111 }
```

APPENDIX B

Database Schema SQL

Listing B.1: Complete database schema (PostgreSQL)

```sql
CREATE TABLE schools (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(150) NOT NULL UNIQUE,
  code        VARCHAR(10)  NOT NULL UNIQUE,
  dean_name   VARCHAR(100),
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE departments (
  id          SERIAL PRIMARY KEY,
  school_id   INTEGER NOT NULL REFERENCES schools(id),
  name        VARCHAR(150) NOT NULL,
  code        VARCHAR(20)  NOT NULL UNIQUE,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE modules (
  id              SERIAL PRIMARY KEY,
  department_id   INTEGER NOT NULL REFERENCES departments(id),
  name            VARCHAR(200) NOT NULL,
  code            VARCHAR(20)  NOT NULL UNIQUE,
  credits         SMALLINT NOT NULL,
  semester        VARCHAR(20)  NOT NULL,
  academic_year   VARCHAR(9)   NOT NULL,
  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
27
28 CREATE TABLE staff (
29   id            SERIAL PRIMARY KEY,
30   school_id     INTEGER REFERENCES schools(id),
31   name          VARCHAR(100) NOT NULL,
32   email         VARCHAR(200) NOT NULL UNIQUE,
33   role          VARCHAR(20)  NOT NULL DEFAULT 'lecturer',
34   password_hash VARCHAR(200) NOT NULL,
35   active        BOOLEAN NOT NULL DEFAULT TRUE,
36   created_at    TIMESTAMP NOT NULL DEFAULT NOW()
37 );
38
39 CREATE TABLE students (
40   id                SERIAL PRIMARY KEY,
41   student_number    VARCHAR(20)  NOT NULL UNIQUE,
42   first_name        VARCHAR(80)  NOT NULL,
43   last_name         VARCHAR(80)  NOT NULL,
44   email             VARCHAR(200) NOT NULL UNIQUE,
45   school_id         INTEGER REFERENCES schools(id),
46   demographic_group VARCHAR(50),
47   active            BOOLEAN NOT NULL DEFAULT TRUE,
48   created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
49   updated_at        TIMESTAMP NOT NULL DEFAULT NOW()
50 );
51
52 CREATE TABLE enrollments (
53   id            SERIAL PRIMARY KEY,
54   student_id    INTEGER NOT NULL REFERENCES students(id),
55   module_id     INTEGER NOT NULL REFERENCES modules(id),
56   academic_year VARCHAR(9)  NOT NULL,
57   status        VARCHAR(20) NOT NULL DEFAULT 'active',
58   created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
59   UNIQUE (student_id, module_id, academic_year)
60 );
61
62 CREATE TABLE esp32_devices (
63   id              SERIAL PRIMARY KEY,
64   device_mac      VARCHAR(17) NOT NULL UNIQUE,
65   room_id         INTEGER REFERENCES rooms(id),
66   firmware_version VARCHAR(20),
67   last_seen_at    TIMESTAMP,
```

```
  status            VARCHAR(20) NOT NULL DEFAULT 'active',
  created_at        TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE rooms (
  id                SERIAL PRIMARY KEY,
  building          VARCHAR(100) NOT NULL,
  room_number       VARCHAR(20)  NOT NULL,
  capacity          SMALLINT,
  esp32_device_id   INTEGER REFERENCES esp32_devices(id),
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE (building, room_number)
);

CREATE TABLE class_sessions (
  id                SERIAL PRIMARY KEY,
  module_id         INTEGER NOT NULL REFERENCES modules(id),
  room_id           INTEGER REFERENCES rooms(id),
  staff_id          INTEGER REFERENCES staff(id),
  scheduled_start   TIMESTAMP NOT NULL,
  scheduled_end     TIMESTAMP NOT NULL,
  session_type      VARCHAR(20) NOT NULL DEFAULT 'lecture',
  status            VARCHAR(20) NOT NULL DEFAULT 'scheduled',
  created_at        TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE image_queue (
  id                SERIAL PRIMARY KEY,
  esp32_device_id   INTEGER REFERENCES esp32_devices(id),
  session_id        INTEGER REFERENCES class_sessions(id),
  image_path        VARCHAR(500) NOT NULL,
  captured_at       TIMESTAMP NOT NULL,
  processed_at      TIMESTAMP,
  status            VARCHAR(20) NOT NULL DEFAULT 'pending',
  error_msg         TEXT,
  created_at        TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE attendance_records (
  id                SERIAL PRIMARY KEY,
```

```
  session_id        INTEGER NOT NULL REFERENCES class_sessions
   (id),
  student_id        INTEGER NOT NULL REFERENCES students(id),
  status            VARCHAR(20) NOT NULL DEFAULT 'present',
  method            VARCHAR(20) NOT NULL DEFAULT '
   face_recognition',
  confidence_score REAL,
  image_queue_id    INTEGER REFERENCES image_queue(id),
  marked_by         INTEGER REFERENCES staff(id),
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE (session_id, student_id)
);

CREATE TABLE student_embeddings (
  id                  SERIAL PRIMARY KEY,
  student_id          INTEGER NOT NULL REFERENCES students(id)
   ,
  embedding           BYTEA NOT NULL,
  model_version       VARCHAR(50) NOT NULL,
  demographic_group   VARCHAR(50),
  active              BOOLEAN NOT NULL DEFAULT TRUE,
  created_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE fairness_metrics (
  id              SERIAL PRIMARY KEY,
  test_date       DATE NOT NULL,
  dataset         VARCHAR(100) NOT NULL DEFAULT 'FairFace',
  race_group      VARCHAR(50)  NOT NULL,
  sample_size     INTEGER,
  detection_rate REAL,
  precision       REAL,
  recall          REAL,
  f1              REAL,
  threshold_used REAL,
  notes           TEXT,
  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE audit_log (
  id            SERIAL PRIMARY KEY,
```

```
  staff_id      INTEGER REFERENCES staff(id),
  action        VARCHAR(50) NOT NULL,
  target_table  VARCHAR(50) NOT NULL,
  target_id     INTEGER,
  old_value     JSONB,
  new_value     JSONB,
  created_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_attendance_session ON attendance_records(
   session_id);
CREATE INDEX idx_attendance_student ON attendance_records(
   student_id, created_at);
CREATE INDEX idx_queue_status       ON image_queue(status,
   created_at);
CREATE INDEX idx_embeddings_model   ON student_embeddings(
   model_version, active);
CREATE INDEX idx_sessions_module    ON class_sessions(
   module_id, scheduled_start);
CREATE INDEX idx_enrollments_lookup ON enrollments(student_id
   , module_id);
```